

Review

Jessica

2026-08-28

```
4 + 3/10^2 # adding and then put a power
```

```
## [1] 4.03
```

```
4 + (3/10)^2
```

```
## [1] 4.09
```

```
(4+3)/10^2
```

```
## [1] 0.07
```

```
((4+3)/10)^2
```

```
## [1] 0.49
```

Remainders

```
37%/%7 # integer part
```

```
## [1] 5
```

```
37%%7 # remainder
```

```
## [1] 2
```

Simple calculation

```
# question 1: compute  
100*(1 + 0.05/12)^24
```

```
## [1] 110.4941
```

```
# question 2: remainder of 3333 divided by 222
3333%%222
```

```
## [1] 3
```

Simple object

```
## Simple objects
x = 2
x <- 2 # assign x to 2 is similar to x= 2 but assigning is better
x <- x + 1 #updated the value of x
x = y = z = 2
# ls() list of the variables assigned
# rm() remove variable or remove all variables: rm(list = ls())
x = y = z = 2
x^y
```

```
## [1] 4
```

```
rm(x) #remove the variable x
```

```
# choose a meaningful name: name cannot be start with number but can end with number
CashOnHand <- 155
Cash_On_Hand <- 155
Cash.On.Hand <- 155
CashOn12 <- 4
```

```
P <- 100
r <- 0.08
n <- 12
t <- 3
Fvalue <- P*(1+r/n)^(n*t) # careful with F because it refers to False
Fvalue # or print(Fvalue)
```

```
## [1] 127.0237
```

```
sprintf("Final Value: $%.2f", Fvalue) # $ for place value
```

```
## [1] "Final Value: $127.02"
```

```
cat(sprintf("Final Value: $%.2f", Fvalue)) # cat to remove the quote
```

```
## Final Value: $127.02
```

Create a vectors

```
## Vector
## Create vectors
1:5 # sequence that increase by 1
```

```
## [1] 1 2 3 4 5
```

```
5:10
```

```
## [1] 5 6 7 8 9 10
```

```
5.1:11.3
```

```
## [1] 5.1 6.1 7.1 8.1 9.1 10.1 11.1
```

```
5:-3 # decreased by 1 until -3
```

```
## [1] 5 4 3 2 1 0 -1 -2 -3
```

```
x <- 1:5 # you can assign a vector with a variable
y <- c(1,100,2,999) # another way to define a vector
```

```
# create a sequence
seq(2, 20, by=2) # start = 2, end=20, increase by 2
```

```
## [1] 2 4 6 8 10 12 14 16 18 20
```

```
seq(2,20, length.out = 100)
```

```
## [1] 2.000000 2.181818 2.363636 2.545455 2.727273 2.909091 3.090909
## [8] 3.272727 3.454545 3.636364 3.818182 4.000000 4.181818 4.363636
## [15] 4.545455 4.727273 4.909091 5.090909 5.272727 5.454545 5.636364
## [22] 5.818182 6.000000 6.181818 6.363636 6.545455 6.727273 6.909091
## [29] 7.090909 7.272727 7.454545 7.636364 7.818182 8.000000 8.181818
## [36] 8.363636 8.545455 8.727273 8.909091 9.090909 9.272727 9.454545
## [43] 9.636364 9.818182 10.000000 10.181818 10.363636 10.545455 10.727273
## [50] 10.909091 11.090909 11.272727 11.454545 11.636364 11.818182 12.000000
## [57] 12.181818 12.363636 12.545455 12.727273 12.909091 13.090909 13.272727
## [64] 13.454545 13.636364 13.818182 14.000000 14.181818 14.363636 14.545455
## [71] 14.727273 14.909091 15.090909 15.272727 15.454545 15.636364 15.818182
## [78] 16.000000 16.181818 16.363636 16.545455 16.727273 16.909091 17.090909
## [85] 17.272727 17.454545 17.636364 17.818182 18.000000 18.181818 18.363636
## [92] 18.545455 18.727273 18.909091 19.090909 19.272727 19.454545 19.636364
## [99] 19.818182 20.000000
```

Using replication

```
rep(1:3, 2) # replication
```

```
## [1] 1 2 3 1 2 3
```

```
rep(1:3, each=2) # specify the replication
```

```
## [1] 1 1 2 2 3 3
```

```
rep(1:3, 1:3) # this one is a bit strange
```

```
## [1] 1 2 2 3 3 3
```

```
# question 4: create vectors
```

```
a <- seq(1, 13, by=2)  
print(a)
```

```
## [1] 1 3 5 7 9 11 13
```

```
b <- rep(seq(1,5,2), seq(5,1,-2))  
b
```

```
## [1] 1 1 1 1 1 3 3 3 5
```

```
# extracting elements of a vector  
x <- c(5,1,6,7,9)  
x[3] # extract the third one
```

```
## [1] 6
```

```
x[1:3]
```

```
## [1] 5 1 6
```

```
x[-4]
```

```
## [1] 5 1 6 9
```

```
x[-c(4,1)] # Remove both 4 and 1st
```

```
## [1] 1 6 9
```

```
# calculation  
x^2
```

```
## [1] 25 1 36 49 81
```

```
y <- x + 1
x^y
```

```
## [1]      15625          1      279936      5764801 3486784401
```

```
# question 5
n <- 10^(0:7)
values <- (1 + 1/n)^n
values
```

```
## [1] 2.000000 2.593742 2.704814 2.716924 2.718146 2.718268 2.718280 2.718282
```

```
# create matrix
cbind(n, values) # column bind
```

```
##           n  values
## [1,] 1e+00 2.000000
## [2,] 1e+01 2.593742
## [3,] 1e+02 2.704814
## [4,] 1e+03 2.716924
## [5,] 1e+04 2.718146
## [6,] 1e+05 2.718268
## [7,] 1e+06 2.718280
## [8,] 1e+07 2.718282
```

```
rbind(n, values) # row bind
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
## n           1 10.000000 100.000000 1000.000000 10000.000000 1.000000e+05
## values      2  2.593742   2.704814   2.716924   2.718146 2.718268e+00
##           [,7]      [,8]
## n           1.00000e+06 1.000000e+07
## values 2.71828e+00 2.718282e+00
```

```
# matrix function
```

```
x <- matrix(1:6, nrow=2, ncol=3)
```

```
# extract value from a matrix
```

```
x[1,2] # gives you the first row and the second column
```

```
## [1] 3
```

```
x[1,] # specify only the row
```

```
## [1] 1 3 5
```

```
x[1:2,]
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

```
x[1:2, -2] # keep the 2 rows and remove the 2nd column
```

```
##      [,1] [,2]
## [1,]    1    5
## [2,]    2    6
```

```
x[1:2, c(1,3)]
```

```
##      [,1] [,2]
## [1,]    1    5
## [2,]    2    6
```

```
# operation on a matrix
```

```
x + x # dimension has to be the same dimension
```

```
##      [,1] [,2] [,3]
## [1,]    2    6   10
## [2,]    4    8   12
```

```
x*x
```

```
##      [,1] [,2] [,3]
## [1,]    1    9   25
## [2,]    4   16   36
```

```
x%*%t(x) #multiplication
```

```
##      [,1] [,2]
## [1,]   35   44
## [2,]   44   56
```

```
rbind(5:9, 12, (9:5)^2, c(2,8,4,7,3))
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    5    6    7    8    9
## [2,]   12   12   12   12   12
## [3,]   81   64   49   36   25
## [4,]    2    8    4    7    3
```

```
x = "GeeksforGeeks" # string
x1 = 255             # integer
x2 = 23.14           # float
```

```
# string print
sprintf("%s is best", x)
```

```
## [1] "GeeksforGeeks is best"
```

```
# integer print  
sprintf("%d is integer", x1)
```

```
## [1] "255 is integer"
```

```
# float print  
sprintf("%f is float", x2)
```

```
## [1] "23.140000 is float"
```